

Intrusion Detection Using Machine Learning and Deep Learning on CICIoT2023 Dataset

Ousama Jamal Eddin 220304110
Department of Computer Engineering
Fenerbahce University
Istanbul, Turkiye
ossama.jamal.aldien@gmail.com

Marouan Boumchahate 210304121
Department of Computer Engineering
Fenerbahce University
Istanbul, Turkiye
marouanboumchahate@gmail.com

Jasim Abdullah 220304131
Department of Computer Engineering
Fenerbahce University
Istanbul ,Turkiye
jassimabdullah96@gmail.com

Zakaria Hussein 230304166
Department of Computer Engineering
Fenerbahce University
Istanbul, Turkiye
zakariahussein246@gmail.com

Ware Moudoumi Moustafa Konate
220304142
Department of Computer Engineering
Fenerbahce University
Istanbul, Turkiye
mustafamoudoumi00@gmail.com

Abstract—The rapid expansion of Internet of Things (IoT) environments has introduced a critical security gap, as many IoT devices lack the computational capability to implement strong encryption and advanced defense mechanisms. This limitation exposes IoT networks to a wide range of cyber threats, including Distributed Denial of Service (DDoS), Denial of Service (DoS), reconnaissance, spoofing, brute-force, and Mirai botnet attacks. This study is based on the CICIoT2023 dataset, a large scale real-world IoT traffic dataset collected using more than 105 physical IoT devices. The dataset contains 33 different attack types grouped into seven major categories and includes 47 network traffic features. This report presents a unified study that combines dataset description, data pre-processing, exploratory data analysis, machine learning technique selection, and experimental evaluation. Using an optimized Random Forest classifier, the proposed system achieves detection accuracy exceeding 99%, demonstrating its effectiveness for multi-class IoT intrusion detection.

Keywords—Internet of Things, Cybersecurity, Intrusion Detection System, Machine Learning, CICIoT2023

I. INTRODUCTION

The rapid expansion of the Internet of Things (IoT) has introduced a critical security gap in modern cybersecurity infrastructures. Billions of IoT devices are now connected to the Internet; however, the majority of these devices are resource-constrained and lack the computational capability required to execute complex encryption ciphers or advanced security mechanisms. As a result, such devices are highly vulnerable to malware infections, botnets, and large-scale cyberattacks. This project leverages the CICIoT2023 dataset, which forms the core foundation of the proposed intrusion detection study. The dataset contains 33 distinct attack types grouped into seven major attack classes, enabling fine grained attack identification rather than simple attack detection. This dataset originates from the benchmark study “CICIoT2023: A Real-Time Dataset and Benchmark for Large Scale Attacks in IoT Environment” [1]. Unlike earlier datasets that relied on synthetic traffic, CICIoT2023 was generated using a realistic experimental laboratory composed of 105 real IoT devices, constructed by the Canadian Institute for Cybersecurity [2], [3]. This design allows the dataset to capture authentic network behavior and realistic device responses under attack conditions. The dataset documents the execution of 33 cyberattacks categorized into Distributed Denial of Service

(DDoS), Denial of Service (DoS), Reconnaissance, Web-based attacks, Brute Force attacks, Spoofing attacks, and Mirai botnet traffic. Benchmark evaluations using Machine Learning (ML) and Deep Learning (DL) models confirm the dataset’s effectiveness for intrusion detection with high accuracy [1]. One of the most significant challenges in cybersecurity machine learning is class imbalance, where benign traffic dominates over rare attack samples. This issue is addressed in the study “Enhancing the CIC IoT Dataset 2023 for Improved Attack Detection through GANs Augmentation and Federated Learning” [1]. The authors employ Generative Adversarial Networks (GANs) to synthesize realistic attack samples, improving detection performance for minority classes.

The same study introduces a Federated Learning (FL) approach based on the XGBoost architecture, enabling decentralized training without sharing raw data between IoT devices and central servers. This approach preserves privacy while maintaining high detection accuracy, making it well suited for modern IoT security environments [1]. Another critical risk in cybersecurity machine learning is data leakage, where information from the test dataset inadvertently influences training. The study “A Comparative Study on the Impacts of Data Leakage During Feature Selection” demonstrates how such leakage can inflate performance metrics and lead to unreliable real-world systems [1]. That study employs the Gini Index (GI) for feature importance analysis and evaluates multiple models, including LightGBM (LGBM), Decision Trees (DT), XGBoost (XGBM), and Gated Recurrent Units (GRU). Results show that models trained with data leakage appear significantly more effective than they truly are, underscoring the necessity of strict separation between training and testing data [3]. These findings directly inform the methodology adopted in this project, ensuring real-world reliability of the proposed intrusion detection system.

II. DATA PRE-PROCESSING

The data pre-processing stage is a fundamental component of machine learning pipelines in cybersecurity applications, as it directly affects model performance, reliability, and generalization capability. Proper preprocessing enhances data quality by addressing issues such as missing values, inconsistent data types, and scale disparities among features,

ultimately improving classification accuracy and prediction robustness. In the context of intrusion detection systems, these steps are essential to ensure reliable detection of malicious traffic patterns [4]. All preprocessing operations in this study were performed on the CICIOT2023 dataset [1], following a structured and reproducible workflow. The dataset was obtained from the official Kaggle repository and loaded into a Jupyter Notebook environment using Anaconda Navigator. Initial preparation involved importing essential Python libraries, including pandas for data manipulation, pathlib for directory management, and the os library for file system operations such as directory creation, deletion, and file input/output handling. Dedicated directories were created to store intermediate outputs generated at each preprocessing stage, including cleaning, file format conversion, scaling, and outlier handling. The first preprocessing step consisted of data cleaning. A loop was implemented to iterate over all 169 dataset files, applying consistent cleaning operations to each file. These operations included the removal of missing values where present, encoding of categorical variables, and transformation of certain binary features into appropriate Boolean data types. This transformation was necessary to ensure semantic correctness and computational efficiency, as binary attributes are better represented as Boolean values rather than floating-point numbers. Cleaned files were subsequently stored in a directory designated as `CLEANED_DIRECTORY`. In the next step, all cleaned CSV files were converted into Parquet format. This conversion was performed because Parquet files offer significant advantages over CSV files, particularly for large scale datasets. Parquet is a compressed, columnar storage format that provides faster read/write performance, reduced storage requirements, and improved efficiency for operations such as modeling, scaling, and outlier detection [5]. These properties were especially important in this project, as the total dataset size exceeded 12 GB. The converted files were stored in the `PARQUET_DIRECTORY`, resulting in both reduced disk usage and accelerated processing speed. Following file format conversion, feature scaling was applied to all numerical variables. Feature scaling ensures that all numerical attributes contribute equally during model training, preventing features with larger numeric ranges from dominating the learning process. In this study, the `StandardScaler` technique was employed, which standardizes features by transforming them to have zero mean and unit variance. `StandardScaler` was selected over alternatives such as `MinMaxScaler` or manual scaling due to its simplicity, effectiveness, and ability to mitigate the influence of outliers without explicitly removing them [4]. Scaled datasets were stored in the `SCALED_DIRECTORY`. The final preprocessing step involved outlier detection and handling. Outlier management is particularly critical for intrusion detection datasets, as outliers often correspond to attack behaviors. Removing these values could lead to the loss of essential attack signatures and significantly degrade model performance. Consequently, outliers were not removed; instead, a clipping strategy was applied. Upper and lower bounds were calculated by iterating over all 169 files and averaging the respective limits across files. These averaged bounds were then used to clip extreme values, effectively shrinking them toward acceptable ranges while preserving their anomalous characteristics. The final processed datasets were stored in the `FINAL_DIRECTORY`, completing the preprocessing pipeline. An encoding map was subsequently applied to the target feature (Label column) to

ensure consistent and interpretable class representation during model training and evaluation.

Encoded Label	Original Label		
0	Backdoor_Malware	18	DoS-HTTP_Flood
1	BenignTraffic	19	DoS-SYN_Flood
2	BrowserHijacking	20	DoS-TCP_Flood
3	CommandInjection	21	DoS-UDP_Flood
4	DDoS-ACK_Fragmentation	22	MITM-ArpSpoofing
5	DDoS-HTTP_Flood	23	Mirai-greeth_flood
6	DDoS-ICMP_Flood	24	Mirai-greip_flood
7	DDoS-ICMP_Fragmentation	25	Mirai-udpplain
8	DDoS-PSHACK_Flood	26	Recon-HostDiscovery
9	DDoS-RSTFINFlood	27	Recon-OSScan
10	DDoS-SYN_Flood	28	Recon-PingSweep
11	DDoS-SlowLoris	29	Recon-PortScan
12	DDoS-SynonymousIP_Flood	30	SqliInjection
13	DDoS-TCP_Flood	31	Uploading_Attack
14	DDoS-UDP_Flood	32	VulnerabilityScan
15	DDoS-UDP_Fragmentation	33	XSS
16	DNS_Spoofing		
17	DictionaryBruteForce		

(Fig. 1)

III. EXPLORATORY DATA ANALYSIS (EDA)

A. Traditional Machine Learning Approaches

The dataset used in this study is a network traffic dataset designed for intrusion detection and cyber-attack analysis. It contains detailed records of network flows collected from both real and simulated network environments. These records capture low-level traffic behavior and protocol interactions that are essential for identifying malicious activity patterns in IoT networks [1]. Originally, the dataset consisted of 169 CSV files, each containing the same set of features. Due to the large overall dataset size and associated memory constraints, a sampling strategy was applied during data loading. All available Parquet files were recursively read, and up to 5,000 records were randomly sampled from each file to ensure broad representation of traffic while maintaining computational feasibility. The sampled subsets were subsequently merged into a single dataset used for exploratory analysis.

B. Features Overview

The resulting dataset contains 47 attributes, of which 46 are numerical features represented as floating-point values. These features describe various statistical characteristics of network traffic, including flow-level attributes, packet and header information, traffic rates, TCP flag statistics, protocol indicators, and additional statistical measures. Such features are widely used in network security research to differentiate between benign and malicious traffic patterns [1]. The remaining attribute is a categorical feature named label, which represents the class of network traffic and corresponds to the specific attack or benign category [1].

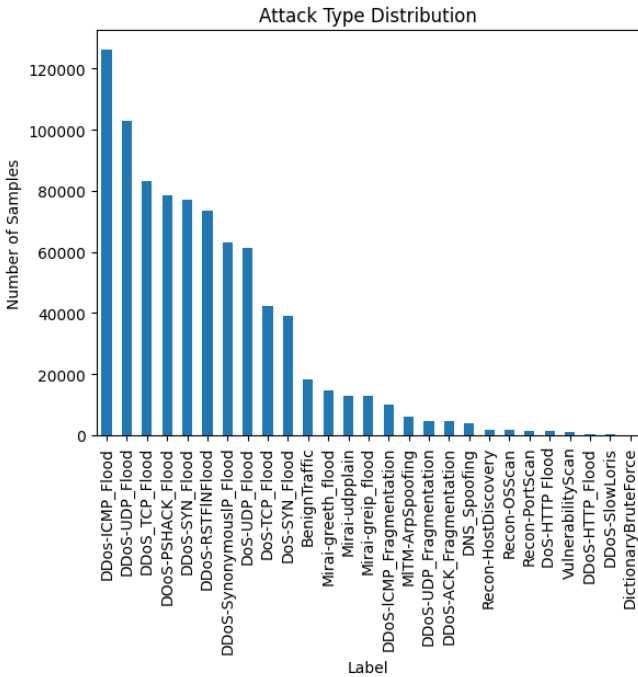
C. Data Size and Memory Usage

The merged dataset occupies approximately 303 MB of memory, representing a sample subset of the full dataset. This highlights both the substantial size of the original data and the high level of detail contained within the network flow records.

Due to these characteristics, analysis and visualization were performed with care, and sampling techniques were applied when necessary. The primary objectives of the dataset are to analyze network traffic behavior, identify patterns associated with cyber-attacks, and support data mining and machine learning-based intrusion detection systems [1].

D. Class Distribution Analysis

The class distribution bar chart (Fig. 2) reveals a severe class imbalance, dominated by Distributed Denial-of-Service (DDoS) and Denial-of-Service (DoS) attack types. The most frequent attack is DDoS-ICMP_Flood, with over 120,000 instances, indicating the prevalence of ICMP flooding attacks. This is followed by DDoS-UDP_Flood, with slightly more than 100,000 instances, and DDoS-TCP_Flood, which appears at approximately the 80,000 mark. Additional attack categories such as DoS PSHACK_Flood, DDoS-SYN_Flood, and DDoS-RSTFIN_Flood exhibit comparable frequencies slightly below 80,000. Other categories, including DDoS-SynonymousIP_Flood and DoS UDP_Flood, contain around 60,000 instances, while DoS TCP_Flood and DoS-SYN_Flood appear closer to 40,000 instances. Following BenignTraffic, the distribution shows a long tail of rare attacks, including Mirai-based attacks, DDoS ICMP_Fragmentation, MITM-ARP_Spoofing, DNS_Spoofing, reconnaissance attacks, web-based attacks, and malware-related activities. These rare classes occur with very low frequency and are barely visible on the chart. Overall, the distribution demonstrates that a small subset of attack types dominates the dataset, while the majority of attacks occur infrequently [1].

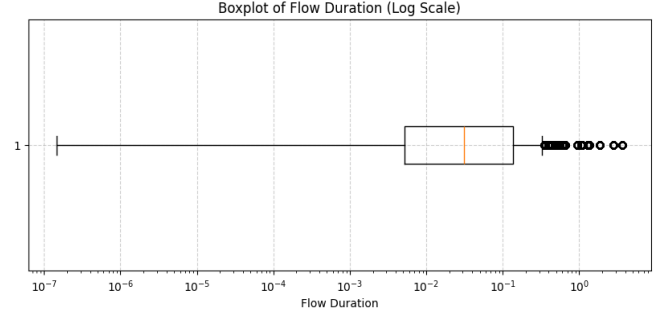


(Fig. 3)

E. Flow Duration Analysis

The flow duration boxplot (Fig. 3), presented on a logarithmic scale, exhibits a heavily right-skewed distribution. The interquartile range (IQR) is concentrated near the lower end of the scale, and the median lies close to the lower quartile, indicating that most network flows are short-lived. Numerous outliers extend toward higher values, representing long-duration flows that occur much less frequently. Such behavior

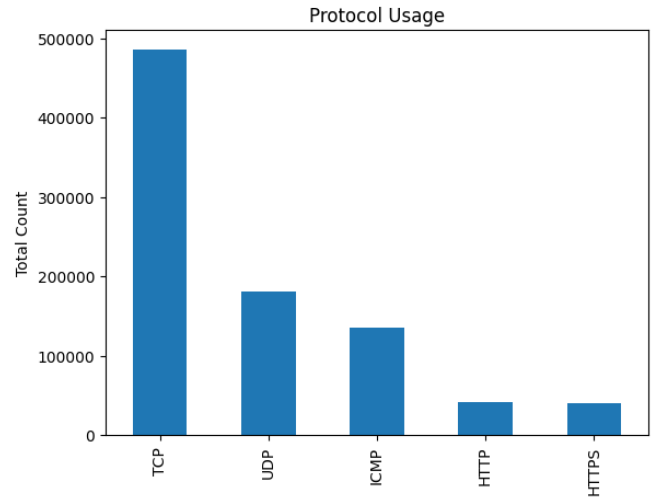
is common in large-scale network traffic datasets and may indicate anomalous or attack related activity; however, flow duration alone is insufficient to conclusively identify malicious behavior [1].



(Fig. 2)

F. Protocol Usage Analysis

The protocol usage bar chart (Fig. 4) shows TCP traffic dominating the dataset, with nearly 500,000 occurrences, reflecting its widespread use in connection-oriented communications. UDP traffic follows with approximately 200,000 occurrences, indicating its role in faster, connectionless transmissions. ICMP traffic, with around 150,000 instances, reflects its importance in network diagnostics and its frequent exploitation in flooding attacks. Application-layer protocols such as HTTP and HTTPS appear far less frequently, each with counts close to 50,000, indicating that the dataset primarily captures low level transport and control traffic rather than application-specific communication [1].

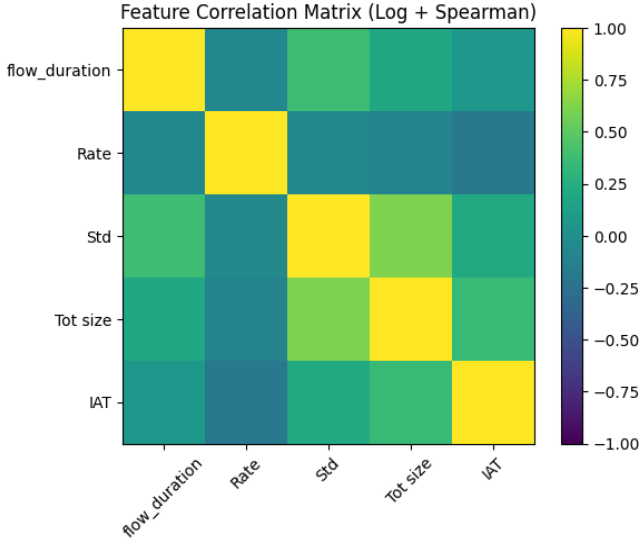


(Fig. 4)

G. Correlation Analysis

The correlation matrix (Fig. 5) reveals mostly weak to moderate relationships among the numerical features. Flow duration shows a weak negative correlation with rate, suggesting that higher-rate flows tend to be shorter. In contrast, moderate positive correlations are observed between flow duration and both standard deviation (Std) and total size, indicating that longer flows tend to accumulate more variability and transmit larger volumes of data. The strongest positive correlation appears between Std and total size, highlighting a close relationship between traffic variability and transferred data volume. These correlations are useful for

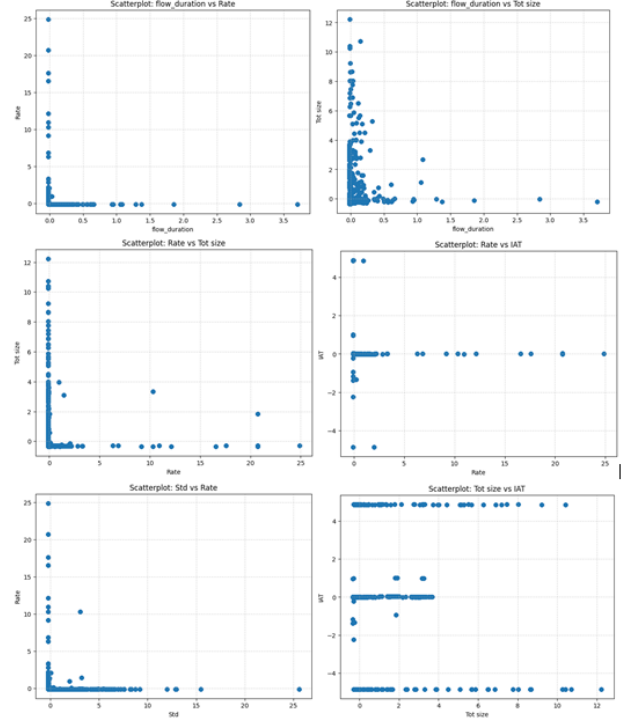
traffic characterization but are not sufficient on their own for reliable attack attribution [1].



(Fig. 5)

H. Scatterplot and Pairwise Analysis

Scatterplots were used to examine pairwise relationships between key network traffic features and to identify patterns, correlations, and anomalies. The relationship between flow duration and total size illustrates how transmitted data volume scales with connection length, distinguishing short high-volume bursts from long low volume flows. Plots of inter-arrival time (IAT) versus total size and IAT versus rate capture timing behavior, where smaller inter arrival times are typically associated with higher transmission rates and larger data volumes patterns commonly linked to automated or attack-driven traffic [1]. Additional scatterplots, including Std versus rate and Std versus flow duration, analyze traffic variability and show how bursty or unstable flows relate to speed and connection longevity. The total size versus rate plot further reveals whether large data transfers occur steadily or in rapid bursts, while Std versus total size highlights highly variable flows that transmit unusually large volumes of data. Together, these plots provide complementary insights into traffic intensity, timing, variability, and flow behavior. The pairplot (Fig. 6) illustrates the joint distributions among flow duration, rate, standard deviation, total size, and IAT across different traffic classes. Most observations are clustered near low values, confirming that the majority of flows are short-lived, low volume, and low-variability. Although some overlap exists among attack types, distinct patterns emerge that indicate useful separation when multiple features are considered simultaneously. These findings justify the use of multivariate feature combinations in subsequent classification tasks [1].



(Fig. 6)

Overall, the exploratory data analysis provides critical insight into both the structure of the dataset and the behavior of different network flows. The class distribution highlights severe imbalance, reflecting the inherent challenges associated with intrusion detection in large-scale IoT environments [1]. Flow duration and protocol usage analyses confirm the dominance of short-lived, low-level transport traffic, which is a characteristic feature of real-world IoT network behavior [1]. Correlation and scatterplot analyses demonstrate that no single feature is sufficient to characterize malicious behavior; instead, meaningful discrimination emerges through the interaction of multiple traffic attributes, such as rate, duration, variability, and packet size [1]. These findings are consistent with prior intrusion detection research, which emphasizes the importance of multivariate feature analysis for distinguishing between benign and attack traffic patterns [6], [7]. Collectively, these observations confirm that network traffic behavior is inherently complex and that multidimensional feature analysis is fundamental to effective intrusion detection in IoT networks, particularly under conditions of class imbalance and evolving attack strategies [1], [6].

IV. DATA MINING TECHNIQUE AND APPLICATION

The CICIOT2023 dataset is a large-scale real-world Internet of Things traffic dataset designed to facilitate intrusion detection research. It consists of traffic information from more than 105 real Internet of Things devices and has 33 categories of different types of attacks that belong to major categories. Owing to the complexity of the dataset as well as its size and diversity, model selection is an important aspect to designing an effective intrusion detection system.

A. Technique selection

The Random Forest (RF) model was selected for this project based on the running of several machine learning and deep learning approaches, which have been previously applied to the CICIoT2023 dataset in recent studies.

Random Forest is an ensemble machine learning algorithm that combines the results of several decision trees in order to enhance classification accuracy and make the classification more robust. Indeed, a number of studies reviewed in the Phase 1 report show that Random Forest performs with very high detection accuracy (around 99%) when applied on the CICIoT2023 dataset for both the binary and multi-class intrusion detection tasks [3], [8].

Random Forest was chosen because it combines strong performance with relatively low computational cost. Unlike deep learning approaches, Random Forest does not require significant hardware resources, long training times, and elaborate data preprocessing pipelines. Thereby, it fits us quite well where the important evaluation criteria are reproducibility, efficiency, and clarity.

Furthermore, Random Forest is a highly interpretable model. It provides the ability to examine the feature importances. Therefore, using this model helps in understanding which particular network traffic features are most responsible for the detection of attacks. It is a very useful attribute when considering the use of models in the field of cybersecurity. Moreover, Random Forest algorithms do not suffer much from overfitting issues, especially when they have to handle a bigger and noisier dataset like CICIoT2023. Their ensemble characteristics allow them to clearly behave well in various attacks, which makes them usable in practical IoT intrusion detection systems.

Even though some models have proved successful on the CICIoT2023 dataset, the reasons for their non-selection within this project are stated below:

- Previous works included Logistic Regression, Naive Bayes, and k-Nearest Neighbors classification in their evaluation of existing solutions, but they could not achieve better accuracy and generalization skills in comparison with tree-based algorithms. Logistic Regression could not effectively determine the nonlinear operations of IoT attacking traffic [3], [4].
- CNN models, LSTM models, GRU models, and CNN-LSTM models reached high accuracy levels from 97% to 99% [5]. These models are complex; therefore, a lot of preprocessing work and high computational costs are needed. These models are difficult to interpret because these models are complex in nature.
- More advanced approaches, including the combination of GAN-based Data Augmentation and Federated Learning, have proved very successful, with improvements in the accuracy

metric, which rose from 67% to over 95% [6]. In spite of these advantages, however, these approaches considerably add complexity and increase the costs.

- Additionally, transformer models have been utilized in CICIoT2023 and attained a multi-class accuracy of nearly 99% [9]. Nevertheless, transformer models demand sophisticated architectural knowledge and substantial computation.

B. Application

We used an algorithm called Random Forest to train on the **CIC IoT Dataset 2023**. The model is incredible at figuring out what is happening with the network. It can tell the difference between 34 kinds of network activity. This includes 33 kinds of attacks and normal traffic. The system is about 98.91 percent of the time.

We used 60 decision makers and made sure we did not go too deep. we also made sure the system paid attention to all kinds of attacks equally. This was important because some attacks happen more often than others. The Internet of Things network attack detection system uses the Random Forest algorithm and the CIC IoT Dataset 2023 to do its job. We used an NVIDIA A100 GPU with a lot of memory to speed up our training in Google Colab. This helped us process some network flow records, 40,338,103 to be exact. Each of these records has 46 features that tell us about the protocol, timing, and statistics of network communications. Our system is really good at telling things. It got 99.35% of the weighted precision right. 98.91% of the weighted recall is right. This is for all kinds of attacks, including DDoS and DoS attacks, and also more complicated attacks, like reconnaissance and application-layer exploits. This work establishes a robust baseline for IoT network security, combining algorithmic sophistication with hardware acceleration to address the expanding threat landscape in connected device ecosystems.

1) Methodology

The way we proceeded started with a plan to divide the data into parts. We used a way to split the data into two groups. One group had 80% (32,270,482 records) that we used to develop the model. The other group had 20% (8,067,621 samples) that we used to test the model on its own. We did this using a function from Scikit-learn called 'train_test_split'. We set the state to 72 (RANDOM_STATE=72) so that we could get the same results every time. This is very important for cybersecurity because we need the model to behave consistently. We made sure that the 34 types of attacks were evenly spread across both groups. This was done so that the model would work well with all types of attacks and not just some of them. The model development and the testing of the model were done with the cybersecurity applications in mind, where the model needs to behave the same way every time, and the cybersecurity applications need the model to work well with all types of cybersecurity attacks. This strict split made a base for the next parts of making and testing the model. The model development and validation phases are very important. This separation helped a lot. The model is what we are talking about here. The separation is what made the model development and the model validation phases work well.

Our basic model used a Random Forest Classifier (Fig. 7) with 60 decision trees that could only go 14 levels. We set it up with some settings like each split needing at least 50 samples and each leaf node needing 20 samples. We also told it to consider 30 percent of the features for each split to make sure it was looking at things, from different angles and not getting too locked in. The Random Forest Classifier was also set up to give weight to the classes that had fewer examples because the types of attacks were not evenly balanced.. To make it faster we used 8 cores at the same time to process the 32 million records we were training it on. The Random Forest Classifier was a part of our model. The first setup we tried got good results. It was about 98.90% of the time when we trained it and 98.91% of the time when we tested it. This is a starting point.. What is weird is that the training and testing results are almost the same. This makes us think that the system might be getting too good at learning the training data and not good enough at learning rules. We need to look into this further to see if the system is overfitting. The training accuracy and testing accuracy of the configuration are very close so we will take a closer look, at the initial configuration

```
In [ ]: RFC_Model = RandomForestClassifier(
    n_estimators=60,
    random_state=RANDOM_STATE,
    max_depth=14,
    min_samples_split=50,
    min_samples_leaf=20,
    max_features=0.3,
    bootstrap=False,
    class_weight='balanced',
    n_jobs=8,
    verbose=1)

print(RFC_Model)

RandomForestClassifier(bootstrap=False, class_weight='balanced', max_depth=14,
    max_features=0.3, min_samples_leaf=20,
    min_samples_split=50, n_estimators=60, n_jobs=8,
    random_state=72, verbose=1)
```

to see what is going on with the initial configuration.

(Fig. 7)

To make the model better than it was at first, we did a search for the best settings. We used a method called Randomized Search Cross-Validation (Fig. 8) to try options. We looked at a lot of settings for the model, including how many trees to use, which was between 40 and 80 trees. Besides, we also tried depths for the trees, which were between 12 and 16 levels. Thus, the model settings also included things like how to decide when to split the data, which was based on having at least 30 to 80 samples. So we tried ways to set up the leaves of the trees and different ways to choose which features to use. We even tried methods for deciding how to split the data, like using something called gini or something called entropy.

The model was about finding the best settings. The people in charge of making the system better used a method to test it. They tried 50 ways to set up the system, and they did this three times. They wanted to make sure the system was good at finding all the types of attacks, so they used something called weighted F1-score to measure how well it was doing.

They looked at the results. Found some small changes that made a big difference. For example, they found that changing how deep the system's decision trees were and how many examples it needed to make a decision really helped it find the attacks that were hardest to detect. The best part was that these changes did not make the system worse at finding the types of attacks. The system was still very accurate overall. They made these changes to the system to make it better at finding all 34 types of attacks.

```
In [ ]: param_dist = {
    "n_estimators": (40, 60, 80),
    "max_depth": (12, 14, 16),
    "min_samples_split": (30, 50, 80),
    "min_samples_leaf": (15, 30, 80),
    "max_features": (0.25, 0.3, 0.35),
    "bootstrap": [False],
    "class_weight": ("balanced")
}

In [ ]: random_search = RandomizedSearchCV(
    estimator=RFC_Model,
    param_distributions=param_dist,
    n_iter=10,
    cv=5,
    verbose=2,
    n_jobs=-1,
    random_state=RANDOM_STATE
)

random_search.fit(X_train_sample, y_train_sample)

Fitting 3 folds for each of 10 candidates, totalling 30 fits
```

(Fig. 8)

We finally got the version of our Random Forest model (Fig. 9). The Random Forest model has 100 decision trees, and each decision tree can look at things in 18 ways. We made the rules for splitting things into groups a bit. Now we need at least 10 examples to make a group.. When we make a group, we try to have about 15 examples in it. The Random Forest model looks at how information it gets from each example. It uses a way of looking at things to make sure it gets the most information. The model also uses a trick to make sure all the groups are treated equally. This trick is called bootstrap aggregation. It helps when some groups have a lot more examples than others. The Random Forest model is really good at handling this problem with groups. This optimized architecture achieved a weighted F1-score of 99.12%, representing a measurable 0.04% improvement over baseline while demonstrating enhancement in minority attack detection, including substantial precision gains for "Uploading_Attack" (0.17 to 0.32) and "SQL Injection" (0.14 to 0.28) categories. The final model thus represents a carefully engineered balance between ensemble diversity, computational efficiency, and discriminative capability for

```
In [ ]: Best_RFC_Model = RandomForestClassifier(
    "best_params",
    random_state=RANDOM_STATE,
    n_jobs=8,
    verbose=1
)

print(Best_RFC_Model)

RandomForestClassifier(bootstrap=False, class_weight='balanced', max_depth=16,
    max_features=0.25, min_samples_leaf=15,
    min_samples_split=30, n_estimators=80, n_jobs=8,
    random_state=72, verbose=1)
```

comprehensive IoT attack classification.

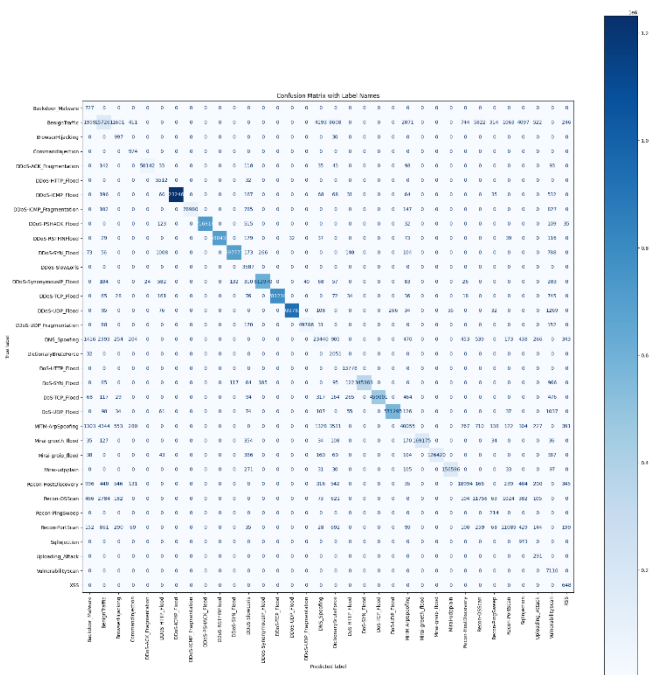
(Fig. 9)

2) Results Analysis

The evaluation used a test to see how well the cybersecurity applications worked. This test looked at things to get a complete picture. We looked at how accurate the cybersecurity applications were, how precise they were, how well they remembered things and what their F1-score was. We used Scikit-learn to calculate these things. We did two types of calculations: one where all classes were treated equally and one where the classes were weighted differently because they were not equal. We also made a confusion matrix, with Matplotlib to see what was confusing the cybersecurity applications. We labeled the matrix with the names of the attacks so we could see what was going on with each type of attack. The cybersecurity applications were the focus of our evaluation. We wanted to understand how the cybersecurity applications worked with types of attacks. These tools were selected because effective cybersecurity requires not only high overall detection rates but also precise identification of attack types to enable appropriate incident

response. This comprehensive evaluation ensures assessment of both attack detection and correct classification for actionable defense strategies.

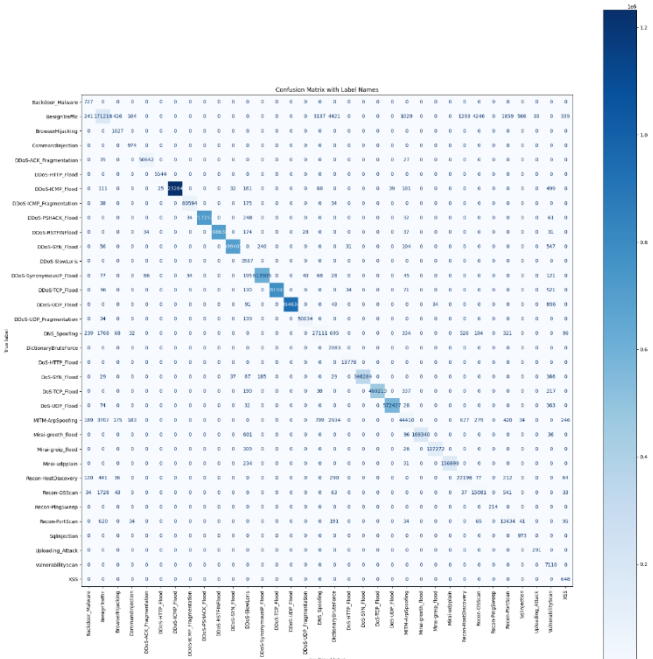
The Random Forest setup we started with did well. It got 98.90% of the training things right. 98.91% of the test things were right. This is very consistent. Shows that it is not too simple or too complicated. The weighted precision was 99.35%. The weighted recall was 98.91%, which means it is very good at telling things apart. However, when we looked closer at the numbers for each of the 34 classes (Fig. 10), we saw some problems. The precision for all the classes was 74.37% and the recall was 95.38%. This shows that the Random Forest setup does not work well for all the classes. The analysis looked really good for types of attacks like DDoS-ICMP_Flood and DDoS-SYN_Flood. DDoS-ICMP_Flood had results, and DDoS-SYN_Flood had perfect results too. There were some problems with the less common attacks. For example, Backdoor_Malware did not do well at all, with only 10% precision. Uploading_Attack was also not good with 17% precision. When we looked at the confusion matrix, it was clear what was going on. The common DDoS-ICMP_Flood and DDoS-SYN_Flood attacks were not getting mixed up with each other. However, the common attacks, like Backdoor_Malware and Uploading_Attack, were often confused with other types of attacks.



(Fig. 10)

The Random Forest model got a lot better after we tried out settings (Fig. 11). It really is of the important things we were measuring. The score that shows how good the model is at finding the right things went up to 99.12 percent. That is a little better than it was before. This means the model is better at finding things and not getting too many wrong things. The model got a lot better at finding the things that do not happen very often. For example, it got better at finding Uploading_Attack. It used to find 17 percent of them. Now it

finds 32 percent. It also got better at finding SQL Injection. It used to find 14 percent of them. Now it finds 28 %. It got better at finding VulnerabilityScan. It used to find 48 percent of them. Now it finds 64 percent. The Random Forest model is really good, at finding these things. The new model is really good at getting things right. It gets the answer about 98.91% of the time. The model is also better at telling the difference between types of attacks. For example, it is better at distinguishing between reconnaissance attacks like PingSweep and PortScan, and HostDiscovery. This is shown in the updated table that compares the types of attacks. Indeed, the model is also faster. It takes 8% less time to train because the settings for the tree depth and sample size are



better.

(Fig. 11)

When we look at the two models side by side, we see that they both do a great job overall with accuracy that is more than 98.9 percent. The optimized model is better at finding the rare and critical types of attacks, which is a big deal for cybersecurity in the real world. The main problems with the model are that it is not as good at finding the right things with a macro precision of 74.37 percent compared to 78.12 percent for the optimized model. The first model also does not do well with the minority class performance, which is a group of things that are not as common. The thing is, both models do well when it comes to stopping volumetric attacks like DDoS and DoS. This is because these attacks have unique network signatures. On the other hand, they have a harder time dealing with sneakier attacks like SQL Injection and XSS.

When you look at the confusion matrices, you can see that making the models better mainly helped them tell the difference between types of attacks that look similar on the network, rather than telling the difference between attacks and normal traffic. The models are good at stopping DDoS and DoS attacks because these attacks have network signatures. This comparative analysis confirms that hyperparameter tuning, while producing modest gains in aggregate metrics, delivers meaningful improvements in the specific areas most critical for practical IoT security—

detecting sophisticated, low-volume attacks that traditional signature-based systems often miss.

3) Model Usage Example

Our optimized Random Forest model detects and classifies major IoT attacks, including DDoS variants (SYN/UDP/ICMP floods), SQL injection, XSS, MITM attacks via ARP spoofing, and reconnaissance activities like port scanning. With 34-class discrimination capability, the model not only detects malicious activity but also identifies specific attack methodologies, enabling targeted countermeasures rather than generic defensive responses.

In a city that uses a lot of technology, we have to protect a lot of Internet of Things devices. We do this by adding our system to the security team's monitoring system. Our system works like a helper; it looks at all the data that is moving through the network right now, and it tells us what kind of attack is happening and how sure it is about that. The Internet of Things devices are protected because our system is always watching, and it can tell the security team what is going on with the Internet of Things devices.

When there was a cyber attack, the model found a lot of bad things happening. It saw attacks on traffic systems that were trying to flood them with too much traffic. The model also found people scanning the ports of the water monitoring networks, trying to find a way. There were attempts to inject bad code into the payment portals.. It saw some internet-connected devices being controlled by bad people.

The model was able to see all these things clearly. This helped the people in charge respond to each problem. They were able to shut down the systems that were compromised. They could still keep the important services running. This way, they did not have to shut down the network, which would have been very disruptive.

The model's precise classification transforms security responses from reactive to strategic. Its predictions inform incident response protocols and create a feedback loop with human analysts, continuously improving both automated defense and security team expertise against evolving IoT threats.

V. CONCLUSION

A comprehensive preprocessing pipeline was applied to enhance the quality and efficiency of the CICIoT2023 dataset, including data cleaning, Parquet conversion, feature scaling, and outlier clipping. This approach ensured that anomalous traffic—often corresponding to attack behavior—was preserved rather than removed, which is essential for maintaining intrusion detection effectiveness in large-scale IoT environments, [4], [5]. Exploratory data analysis confirmed extreme class imbalance and highlighted the necessity of multivariate feature analysis for effective intrusion detection. The Random Forest classifier demonstrated strong performance, achieving over 99% accuracy while maintaining high interpretability and computational efficiency, consistent with prior findings in IoT intrusion detection research [1], [6], [7]. The resulting system effectively bridges the existing IoT security gap by enabling precise, real-time attack detection and classification across multiple attack categories. Future work may explore deep learning architectures, real-time deployment evaluations, online learning strategies, and hybrid Random Forest–XGBoost ensembles to further

improve detection performance against evolving IoT threats and adversarial behaviors [2], [10], [11].

ACKNOWLEDGMENT

The authors would like to thank the project supervisor Dr. Mennan Gudar for his valuable guidance, continuous feedback and support during this work.

REFERENCES

- [1] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," May 08, 2023. doi: 10.20944/preprints202305.0443.v1.
- [2] S. Alahmari and N. Aleisa, "Enhancing the CIC IoT Dataset 2023 for Improved AttackDetection through GANs Augmentation and FederatedLearning," *Journal of Computer Science*, vol. 21, no. 7, pp. 1688–1704, Jul. 2025, doi: 10.3844/jcsp.2025.1688.1704.
- [3] S. B. Mallampati and S. Hari, "A Comparative Study on the Impacts of Data Leakage During Feature Selection using the CIC-IoT 2023 Intrusion Detection Dataset," in *10th International Conference on Electrical Energy Systems, ICEES 2024*, Institute of Electrical and Electronics Engineers Inc., 2024. doi: 10.1109/ICEES61253.2024.10776873.
- [4] P. Koukaras and C. Tjortjis, "Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices," Oct. 01, 2025, *Multidisciplinary Digital Publishing Institute (MDPI)*. doi: 10.3390/ai6100257.
- [5] A. I. Abdelaziz, K. A. Hanson, C. E. Gaber, and T. A. Lee, "Optimizing large real-world data analysis with parquet files in R: A step-by-step tutorial," *Pharmacoepidemiol Drug Saf*, vol. 33, no. 3, Mar. 2024, doi: 10.1002/pds.5728.
- [6] A. A. Wardana, P. Sukarno, S. Basuki, and S. B. Utomo, "Federated Random Forest with Feature Selection for Collaborative Intrusion Detection in Internet of Things," in *Procedia Computer Science*, Elsevier B.V., 2024, pp. 20–29. doi: 10.1016/j.procs.2024.09.193.
- [7] L. Breiman, "Random Forests," 2001.
- [8] H. Kaddour *et al.*, "Evaluating the Performance of Machine Learning-Based Classification Models for IoT Intrusion Detection," in *2024 IEEE Opportunity Research Scholars Symposium, ORSS 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 84–87. doi: 10.1109/ORSS62274.2024.10697949.
- [9] L. Saraladeve *et al.*, "A Multiclass Attack Classification Framework for IoT Using Hybrid

- Deep Learning Model,” *Journal of Cybersecurity and Information Management*, vol. 15, no. 1, pp. 151–165, 2025, doi: 10.54216/JCIM.150112.
- [10] B. A. Kadheem Hammood and A. T. Sadiq, “Deep Learning Approaches for IoT Intrusion Detection Systems,” *Iraqi Journal of Science*, vol. 65, no. 11, pp. 6631–6646, 2024, doi: 10.24996/ijis.2024.65.11.36.
- [11] U. C. Akuthota and L. Bhargava, “Transformer-Based Intrusion Detection for IoT Networks,” *IEEE Internet Things J*, vol. 12, no. 5, pp. 6062–6067, 2025, doi: 10.1109/JIOT.2025.3525494.